FLOWERS-GNNs - Adapting FLOWERS for Irregular Mesh PDE simulations

Thesis Preparation Report
Masters Data Science

Soham Roy
University of Basel

27 May 2026

Abstract

The FLOWERS architecture [2], developed in our group, replaces Fourier and attention-based spatial mixing with learned coordinate warps (pullbacks). It achieves excellent performance on a broad suite of 2D and 3D time-dependent PDE benchmarks. However, its current implementation relies on differentiable interpolation over regular spatial grids to sample displaced features. This thesis investigates whether FLOWERS can be adapted for irregular meshes by framing the continuous "warp" as a dynamic, differentiable routing mechanism over unstructured point clouds. This preparation phase explores the adaptation of the FLOWERS architecture to irregular mesh-based physics simulations by comparing it with the MeshGraphNets (MGN) baseline on fluid dynamics datasets. FLOWERS was originally designed for regular grid-based PDE simulations using learned spatial warps instead of expensive attention or Fourier operators, while MeshGraphNets operates directly on unstructured meshes using message passing[1]. As an initial step, the MGN baseline [1] was mostly reproduced and trained on the cylinder flow dataset up to roughly 250k training steps. Although the original MeshGraphNets paper[1] trains models for significantly longer schedules, the reproduced baseline already achieved stable rollout behaviour and low prediction error with gradual promising lower root mean square error for all time steps with incremental training steps. In parallel, a first version of a FLOWERS-inspired mesh adaptation pipeline was implemented by converting irregular mesh simulations into fixed regular grids suitable for FLOWERS-style processing. Different interpolation and rollout strategies were explored, and early experiments showed promising autoregressive rollout performance for short and medium prediction horizons. The work serves as a practical foundation toward a future FLOWER- GNN architecture capable of efficient long-range transport modelling on irregular meshes.

1 Introduction

Neural surrogate models for partial differential equations (PDEs) have become an important direction for accelerating physical simulations [6, 5]. Instead of solving the governing equations repeatedly with a numerical solver, these models learn an approximation of the time-evolution

operator from data. Given a physical state u_t, the model learns a mapping of the form

$$\mathcal{F}_\theta : u_t \mapsto u_{t+1},$$

where $\mathcal{F}_\theta$ is parameterized by a neural network.

A central challenge is that physical simulations are represented in different spatial formats. Many neural PDE models, including Fourier Neural Operators and FLOWERS, are naturally designed for structured grids [5, 2, 8], where the state is stored as dense tensors over regular spatial coordinates. However, many engineering and scientific simulations use irregular meshes to represent complex geometries efficiently. In such settings, the spatial domain is represented by nodes and connectivity rather than pixels or voxels.

MeshGraphNets [1] address this problem by operating directly on irregular meshes using graph neural network message passing. This makes MGN suitable for datasets such as CylinderFlow and Airfoil, where simulation meshes are highly irregular. However, message passing is local [3, 4] as information only travels from one node to its immediate neighbours in each processor layer. Therefore, modelling fast physical transport or long-range dependencies may require many message-passing steps, increasing computational cost and potentially leading to over-smoothing.

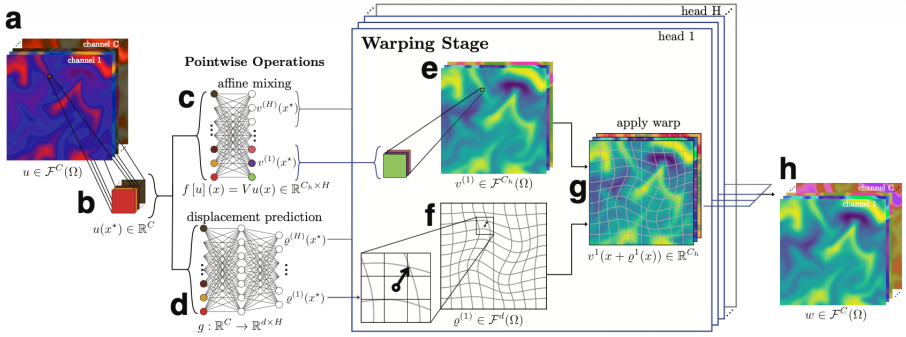

Figure 1. The multi-head SELFWARP layer. (a) Given an input feature map u with C channels, (b) for each pixel x in the image the layer computes (c) a local per-head interaction $v^{(h)}(x) \in \mathbb{R}^{C_h}$ via a linear (affine) projection of the input $u(x)$ and (d) a displacement field (one per head) $\varrho^{(h)}(x)$, also pointwise, using a small MLP. Each C_h-channel per-head interaction field $v^{(h)}$ (e) is warped along the same per-head displacement field $\varrho^{(h)}$ (f) to compute the warped head output (g). The outputs of all heads are concatenated at the output (h).

Figure 1: Flowers architecture [2]

FLOWERS [2] takes a different approach. Instead of using Fourier layers, attention, or deep stacks of local convolutions, FLOWERS learns spatial warps. Each layer predicts displacement fields and samples features at displaced coordinates. On regular grids this is efficient because differentiable interpolation is straightforward. The main question of this thesis is whether this transport-based idea can be adapted to irregular meshes.

The technical objective is therefore to investigate a FLOWERS-GNN hybrid: a model that keeps the transport-based pullback mechanism of FLOWERS, but replaces regular-grid sampling with differentiable interpolation on irregular meshes. This requires solving two coupled problems: first, locating where a displaced point lies in an irregular mesh, and second, interpolating features from nearby nodes or from the containing simplex. This preparation report studies the baseline reproduction of MGN and an initial FLOWERS wrapper based on mesh-to-grid conversion.

2 Literature Review

2.1 Neural PDE Surrogate Models

Neural PDE surrogate models aim to approximate the solution operator of time-dependent physical systems. For a continuous field $u(x,t)$, the goal is to learn an operator that advances the state in time:

$$u(x, t + \Delta t) \approx \mathcal{F}_\theta(u(x,t)).$$

This idea appears in several forms, including Physics-Informed Neural Networks (PINNs) [6], Fourier Neural Operators (FNOs) [5], graph-based simulators such as MeshGraphNets [1], and recent autoregressive neural emulation benchmarks such as APEBench [7].
Fourier Neural Operators perform global spatial mixing in Fourier space. A field is transformed into frequency space [5],

$$\hat{u}(k) = \mathcal{F}(u(x)),$$

and learned spectral weights are applied to selected Fourier modes. This allows efficient long-range communication on regular grids. However, Fourier methods depend strongly on structured spatial discretizations, making them less natural for irregular meshes and complex geometries.

2.2 MeshGraphNets and Message Passing on Irregular Meshes

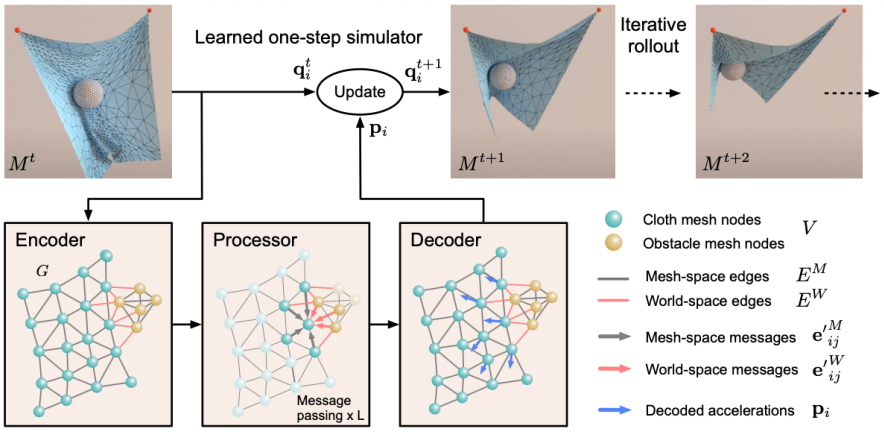

Figure 2: Overview of the MeshGraphNets [1] architecture.

MeshGraphNets [1] represent a physical simulation mesh as a graph

$$G = (V, E),$$

where mesh nodes are graph vertices and mesh edges define graph connectivity. Each node carries physical features such as velocity, pressure, displacement, or node type. A generic message-passing update can be written as

$$m_{ij}^{(l)} = \phi_e(h_i^{(l)}, h_j^{(l)}, e_{ij}),$$

$$h_i^{(l+1)} = \phi_v \left(h_i^{(l)}, \sum_{j \in \mathcal{N}(i)} m_{ij}^{(l)} \right),$$

where h_i is the node embedding, e_{ij} is the edge feature, and $\mathcal{N}(i)$ is the neighbourhood of node i. This formulation follows the general Message Passing Neural Network framework introduced in [4].

This formulation is well suited for irregular geometries because it does not require a Cartesian grid. However, the information flow is local. One message-passing step only moves information by one graph edge. Therefore, if two spatially distant nodes need to exchange information, many processor steps are required [3]. This is one of the main motivations for studying learned transport mechanisms that can route information non-locally. Graph convolutional methods such as GCNs [10] form an important early class of graph neural networks which was also referred to and used for comarisons in various other research.

2.3 FLOWERS: Pullbacks and Multi-Headed Warps

FLOWERS replaces Fourier layers, attention, and heavy convolutional spatial mixing with learned coordinate warps [2].. The key operation is a learned displacement field. For a spatial coordinate x, a FLOWERS head predicts a displacement

$$\rho^{(h)}(x),$$

and samples features at the displaced coordinate

$$x' = x + \rho^{(h)}(x).$$

This is a pullback operation. If ϕ is a coordinate transformation, then the pullback of a field f is

$$(\phi^* f)(x) = f(\phi(x)).$$

In FLOWERS, the transformation is

$$\phi(x) = x + \rho^{(h)}(x),$$

so the model samples

$$f(x + \rho^{(h)}(x)).$$

The model uses multiple heads:

$$\rho^{(1)}(x), \rho^{(2)}(x), \ldots, \rho^{(H)}(x),$$

where each head learns a different transport direction or interaction pattern. This is similar in spirit to multi-head attention, but instead of learning token-to-token attention weights, FLOWERS learns spatial transport pathways.

This design is especially relevant for advection-dominated systems, where physical information is transported through space. Instead of repeatedly passing messages locally, a warp can directly

sample from a physically relevant upstream or displaced position.

2.4 Irregular Mesh Interpolation and the FLOWERS-GNN Problem

The main difficulty in extending FLOWERS to meshes is interpolation. On a regular grid, the displaced coordinate $x + \rho^{(h)}(x)$ can be sampled using standard bilinear interpolation. On an irregular mesh, however, this displaced coordinate will almost never coincide exactly with an existing mesh node.
Therefore, a FLOWERS-GNN must solve the differentiable irregular routing problem. Given a query point

$$q = x + \rho^{(h)}(x),$$

the model must identify nearby mesh structure and interpolate features from surrounding nodes. One possible approach is barycentric interpolation. For a triangle with vertices v_1, v_2, v_3, a point q inside the triangle can be expressed as

$$q = \lambda_1 v_1 + \lambda_2 v_2 + \lambda_3 v_3,$$

with

$$\lambda_1 + \lambda_2 + \lambda_3 = 1.$$

The interpolated feature is then

$$f(q) = \lambda_1 f(v_1) + \lambda_2 f(v_2) + \lambda_3 f(v_3).$$

Another possible approach is inverse-distance weighting:

$$f(q) = \frac{\sum_i w_i f_i}{\sum_i w_i}, \qquad w_i = \frac{1}{\|q - x_i\|^p}.$$

The future thesis work will compare such interpolation strategies, including simplex-based barycentric interpolation, k-nearest neighbour interpolation, and differentiable scattered sampling [9].

3 Methodology

3.1 MeshGraphNets Baseline Reproduction

The first part of the work reproduced the official MeshGraphNets implementation on the CylinderFlow dataset [1]. The model was trained using the original TensorFlow-based DeepMind codebase. The reproduced configuration followed the standard MGN cylinder-flow setup:

$$\text{latent size} = 128,$$

$$\text{message passing steps} = 15,$$

$$\text{output size} = 2.$$

The output corresponds to the two velocity components (v_x, v_y).
The model predicts velocity changes rather than absolute velocity states. If v_t is the current velocity, the model predicts

$$\Delta v_t = v_{t+1} - v_t,$$

and the next state is reconstructed using a residual update:

$$\hat{v}_{t+1} = v_t + \widehat{\Delta v_t}.$$

This residual prediction is important because physical states often change smoothly between consecutive timesteps.
The baseline was trained up to approximately 250,000 training steps. The original MGN paper [1] trains for a substantially longer schedule which is till 5 million steps with a constant learning rate and then with a rate decay till 10 million steps, but the reproduced model already showed stable rollout behaviour becoming more accurate gradually with more training steps.

3.2 FLOWERS Mesh Wrapper

The second part of the work implemented a first [2] FLOWERS-based wrapper for the Cylinder-Flow data. Since the original FLOWERS architecture operates on regular grids, the irregular mesh data first had to be converted into structured tensors.
The pipeline was:

$$\text{irregular mesh} \rightarrow \text{regular grid} \rightarrow \text{FLOWERS} \rightarrow \text{mesh/grid rollout}.$$

The input mesh velocity field had shape

$$(N, 2),$$

where N is the number of mesh nodes and the two channels represent (v_x, v_y). The mesh field was interpolated onto a fixed grid of resolution

$$64 \times 256.$$

The resulting grid tensor had shape

$$(C, H, W) = (2, 64, 256).$$

The wrapper used linear interpolation from mesh nodes to grid cells, nearest-neighbour fallback in sparse regions, and a fixed cylinder obstacle mask. Then for grid to mesh in the output pass, we used torch grid sampling for differentiable grid-to-mesh interpolation.

3.3 Model Size and Architecture

The original FLOWERS paper reports scaled models up to approximately 155.8M parameters. In contrast, this preparation-phase implementation used a much smaller model because the

objective was architectural feasibility rather than large-scale benchmarking.
The main FLOWERS wrapper configuration used:

$$\text{lifting dimension} = 96,$$

$$\text{number of heads} = 32,$$

$$\text{number of groups} = 32,$$

$$\text{number of levels} = 4.$$

A smaller stability-oriented variant was also tested using:

$$\text{lifting dimension} = 48, \qquad \text{number of heads} = 16.$$

The larger wrapper learned faster but showed stronger overfitting on the limited single-trajectory dataset, while the smaller model was more regularized but less expressive.

3.4 Normalization and Loss

To make the FLOWERS wrapper more comparable to MGN, the model was modified to predict velocity deltas rather than absolute next states. The target was

$$y_t = v_{t+1} - v_t.$$

Input velocities and target deltas were normalized separately:

$$\tilde{v}_t = \frac{v_t - \mu_v}{\sigma_v},$$

$$\widetilde{\Delta v_t} = \frac{\Delta v_t - \mu_{\Delta v}}{\sigma_{\Delta v}}.$$

The loss was computed using masked mean squared error over physical update nodes:

$$\mathcal{L} = \frac{1}{|\Omega_{\text{valid}}|} \sum_{i \in \Omega_{\text{valid}}} \left\| \widehat{\Delta v_i} - \Delta v_i \right\|^2.$$

The mask excluded boundary and obstacle nodes, matching the idea used in MGN where only normal and outflow nodes contribute to the velocity loss.

3.5 Autoregressive Rollout Evaluation

Both MGN and FLOWERS were evaluated autoregressively. Starting from an initial state v_0, the model predicts a sequence:

$$\hat{v}_1, \hat{v}_2, \ldots, \hat{v}_T.$$

At each step, the previous prediction is fed back into the model:

$$\hat{v}_{t+1} = \hat{v}_t + \widehat{\Delta v_t}.$$

Rollout performance was measured using MSE and RMSE:

$$\mathrm{MSE} = \frac{1}{N} \sum_{i} \left(v_i - \hat{v}_i \right)^2 ,$$

$$\mathrm{RMSE} = \sqrt{\mathrm{MSE}}.$$

Autoregressive rollout evaluation is widely used in neural PDE benchmarks such as APEBench [7].

4 Results

4.1 Rollout Comparison

Table 1 compares rollout RMSE values for the paper MGN baseline, the reproduced MGN baseline, and the current FLOWERS wrapper. The reproduced MGN was trained up to approximately 250,000 steps, while the FLOWERS wrapper was trained for 20 epochs on the current exported dataset.

Table 1: Rollout RMSE comparison between reported MGN, reproduced MGN, and the current FLOWERS wrapper. The reported MGN column should be filled using exact values from the original paper if available.

Rollout Horizon	**MGN [1]**	**MGN (250k steps)**	**FLOWERS Wrapper**
1 step	0.0023	0.0096	0.0554
10 steps	–	0.0241	0.0847
20 steps	–	0.0303	0.0939
50 steps	0.007	0.0464	0.1060
100 steps	–	0.0660	0.1210
200 steps	0.041	0.0864	0.1430

The reproduced MGN baseline achieved substantially lower rollout error than the current FLOWERS wrapper. This is expected because MGN operates directly on the native irregular mesh, while the current FLOWERS wrapper depends on mesh-to-grid interpolation and is still an early feasibility implementation.
However, the FLOWERS wrapper still produced stable autoregressive rollouts and meaningful predictions across multiple horizons. This suggests that the transport-based FLOWERS mechanism can learn useful dynamics even after projecting irregular mesh data onto structured grids.

5 Conclusion and Future Work

This preparation phase established two important baselines. First, the MeshGraphNets cylinder-flow baseline was reproduced and trained successfully up to approximately 250,000 training steps. Second, a first FLOWERS-based wrapper was implemented for irregular mesh data through mesh-to-grid conversion.

The main technical problem of the thesis is now clearer: FLOWERS-style pullbacks are natural on regular grids but nontrivial on irregular meshes. On a grid, sampling at

$$x + \rho^{(h)}(x)$$

can be handled by bilinear interpolation. On a mesh, the same coordinate may lie inside an arbitrary triangle or near a set of irregularly positioned nodes. Therefore, the central research problem is to design a differentiable routing and interpolation operator for irregular mesh domains.
The next stage will investigate:

- point-location methods for finding the simplex containing a displaced point,
- building a native Irregular mesh in Flowers architecture
- barycentric interpolation inside mesh triangles,
- k-nearest-neighbour and inverse-distance-weighted interpolation,
- differentiable routing over unstructured point clouds,
- multi-trajectory FLOWERS training on the newly exported dataset,
- comparison of rollout stability against the MGN and other neural PDE architectures' baseline.

The long-term thesis objective is to develop a FLOWERS-GNN hybrid architecture [2, 1] that preserves the transport-based inductive bias of FLOWERS while operating directly on irregular meshes. If successful, this could reduce the need for deep local message passing and allow faster long-range physical transport modelling on complex geometries.

References

[1] T. Pfaff, M. Fortunato, A. Sanchez-Gonzalez, and P. Battaglia, Learning Mesh-Based Simulation with Graph Networks, *International Conference on Learning Representations (ICLR)*, 2021. https://doi.org/10.48550/arXiv.2010.03409

[2] T. Muser, A. Spitzer, Matti Lassas, Maarten V. de Hoop, Ivan Dokmanić Flowers: A Warp Drive for Neural PDE Solvers, *arXiv preprint arXiv:2603.04430*, 2026. https://doi.org/10.48550/arXiv.2603.04430

[3] D. Brandstetter, M. Welling, and Daniel Worrall„ Message Passing Neural PDE Solvers, *arXiv preprint arXiv:2202.03376*, 2022. https://doi.org/10.48550/arXiv.2202.03376

[4] J. Gilmer, S. Schoenholz, P. Riley, O. Vinyals, and G. Dahl, Neural Message Passing for Quantum Chemistry, *International Conference on Machine Learning (ICML)*, 2017.https://doi.org/10.48550/arXiv.1704.01212

[5] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, Fourier Neural Operator for Parametric Partial Differential Equations, *International Conference on Learning Representations (ICLR)*, 2021. https://doi.org/10.48550/arXiv.2010.08895

[6] M. Raissi, P. Perdikaris, and G. E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *Journal of Computational Physics*, 378:686–707, 2019. https://doi.org/10.1016/j.jcp.2018.10.045

[7] F. Koehler, S. Niedermayr, R. Westermann, and N. Thuerey, APEBench: A Benchmark for Autoregressive Neural Emulators of PDEs, *Advances in Neural Information Processing Systems (NeurIPS)*, 38, 2024. https://doi.org/10.48550/arXiv.2411.00180

[8] Polymathic AI, The Well: A Large-Scale Collection of Diverse Physics Simulations for Machine Learning, *https://polymathic-ai.org/the_well/*, accessed 2026.

[9] J. A. Bærentzen, Jens-Gravesen and collaborators, Guide to computational geometry processing. Foundations, algorithms, and methods DOI:10.1007/978-1-4471-4075-7 *Guide to Computational Geometry Processing*, January 2012.

[10] T. N. Kipf and M. Welling, Semi-Supervised Classification with Graph Convolutional Networks, *International Conference on Learning Representations (ICLR)*, 2017. https://doi.org/10.48550/arXiv.1609.02907